

6장 늦게 도착한 이벤트를 셀 것인가 버릴 것인가

5주차 · 스트리밍 처리

2장에서 이벤트 시간과 처리 시간이 벌어진다는 것을 확인했고, 5장은 늦게 도착한 이벤트를 이미 마감한 집계에 반영할 것인가라는 질문을 남겼다. 이 장은 그 판단을 사람의 재량이 아니라 스트리밍 엔진이 따르는 규칙으로 바꾸고, 수용된 이벤트와 폐기된 이벤트가 각각 어디에 증거를 남기는지 실제 실행으로 확인한다.

이 장에 나오는 수치는 민원 이벤트 14건을 파일 세 개로 나누어 흘려 넣은 실습 6.1의 실행 산출물이다.

1. 오늘의 질문

- 끝이 없는 데이터에서 "최근 10분에 몇 건"을 세려면 어디를 잘라야 할까?
- 22:04에 발생한 민원이 22:00~22:10 집계가 끝난 뒤에 도착하면, 세야 할까 버려야 할까?
- 그 기준을 무엇으로 정하고, 버린 건수는 어디서 확인할까?
- 집계 위에 알림을 얹으면 무엇이 새로 깨질까?

2. 창구 마감으로 보는 이 장

이 장의 네 개념은 은행이나 구청 민원실 창구의 하루에 하나씩 대응한다.

창구	스트리밍 처리
접수 대장의 줄이 하루 종일 늘어난다	스트림은 행이 계속 추가되는 무한 테이블이다
시간대 칸으로 나누어 접수 건수를 센다	이벤트 시간 기준으로 시간 창을 잘라 센다
마감 시각에 곧바로 닫지 않고 규칙만큼 더 기다린다	watermark가 지연을 얼마나 허용할지 정한다
유예를 넘겨 온 서류는 접수하지 않고 반려 건수만 적는다	폐기된 이벤트는 집계에 안 남고 카운터에만 남는다
대기 인원이 기준을 넘으면 창구를 늘린다	창의 건수가 임계를 넘으면 알림을 보낸다

비유가 깨지는 지점은 마감 시각을 무엇이 정하느냐다. 창구 직원은 벽시계를 보고 닫지만, watermark는 시계가 아니라 **지금까지 도착한 데이터**에서 계산된다. 이벤트가 하나도 들어오지 않으면 실제 시각이 아무리 흘러도 창은 닫히지 않는다.

3. 스트림은 끝없이 자라는 표다

도착하는 대로 처리하는 코드를 처음부터 새로 배우려면 부담이 크다. Spark Structured Streaming의 발상은 그 부담을 없애는 쪽이다. 스트림을 "행이 계속 추가되는, 끝이 없는 테이블"로 간주하고 이미 아는 테이블 질의를 그 위에 그대로 얹는다. 배치에서 "지역구별 민원 건수"를 세던 질의를 고치지 않고 쓴다.

달라지는 것은 실행 방식이다. 강남구 민원이 한 건 더 도착했을 때 엔진은 전체를 처음부터 다시 세지 않고, 이미 들고 있던 "강남구 2"라는 중간 결과에 1을 더해 3으로 고친다. 이 중간 결과를 **상태(state)**라고 부른다. 상태가 있어서 증분 계산이 되지만, 상태는 쌓이므로 "언제 버릴 것인가"라는 질문이 따라온다. 그 답이 5절의 watermark다.

엔진은 흐름을 아주 잘게 쪼갠 작은 배치의 연속으로 처리하며, 이 단위를 **micro-batch**라고 한다. 4장에서 직접 돌렸던 폴링 루프(poll→처리→커밋)와 구조가 닮았는데, 다만 루프를 사람이 아니라 엔진이 돌린다. 검증된 배치 실행 엔진을 재사용하고 "이 배치를 처리했는가"를 단위로 실패를 복구할 수 있는 대신, 최소 지연이 배치 주기만큼 생긴다. 초에서 분 단위면 충분한 공공 대시보드에서는 감수할 만한 절충이다.

micro-batch를 언제 돌릴지는 **trigger**가 정한다. 지정하지 않으면 앞 배치가 끝나는 대로 다음 배치를 돌려 지연을 최소화하고, `processingTime="1 minute"` 으로 두면 1분마다 돌아 대시보드 갱신 주기에 맞춘다.

`availableNow` 는 쌓인 데이터를 모두 처리한 뒤 종료하므로, 스트리밍 코드로 정기 증분 작업을 구현하는 셈이 되어 7장의 배치와 만난다.

무엇을 내보낼지는 **output mode**가 따로 정한다. `complete`는 결과 표 전체를 매번 내보내고, `update`는 이번 배치에서 바뀐 행만, `append`는 다시는 바뀌지 않을 확정 행만 내보낸다. `update` 모드에서는 같은 키(창+지역구)의 행이 여러 번 나오므로 받는 쪽 저장소가 5장의 업서트(upsert)로 받아야 하고, `append` 모드는 "이 창은 다시 안 바뀐다"고 판단할 근거가 필요하므로 watermark 없이는 집계에 쓸 수 없다.

같은 스트림에 같은 질의를 걸어도 이 선택 하나로 산출물의 성격이 바뀐다. 재난 상황판의 "지금까지의 지역별 신고 건수"는 계속 갱신되어야 하니 `update`나 `complete`이고, 사후 감사용으로 남기는 "확정된 10분 단위 집계"는 불변 파일이어야 하니 `append`와 watermark를 함께 쓴다. 어떤 수치가 갱신되는 잠정치이고 어떤 수치가 확정 기록인지 산출물마다 적어 두지 않으면, 나중에 "대시보드와 보고서가 왜 다른가"라는 질문에 설계로 답할 수 없다.

4. 시간 창 — 겹치지 않게 자를까, 밀면서 겹칠까

무한 테이블에서 지역구별 건수를 그냥 세면 그 수는 시스템이 켜져 있는 한 계속 증가한다. 운영에서 필요한 것은 "지금까지 전부"가 아니라 "최근 10분에 몇 건"이므로, 이벤트를 시간 창으로 잘라 창 안에서 센다. 이때 기준 시각은 도착 시각이 아니라 **발생 시각(이벤트 시간)**이어야 지연 도착한 민원도 발생한 시간대의 통계에 들어간다.

창을 만드는 방식은 두 가지다. **tumbling window**(구르는 창)는 시간축을 겹치지 않게 자르며, 22:00–22:10과 22:10–22:20처럼 벽시계의 10분 칸을 그대로 쓴다. 모든 이벤트가 정확히 한 창에 속하므로 정기 보고와 확정 통계에 맞는다. **sliding window**(미끄러지는 창)는 창 크기는 같지만 일정 간격으로 창을

밀어 겹치게 만든다. 크기 10분에 슬라이드 5분이면 21:55-22:05, 22:00-22:10, 22:05-22:15로 이어지고, 한 이벤트가 정확히 두 창에 속한다(창 크기 ÷ 슬라이드 = 2).

어느 창에 속하는지는 손으로도 계산할 수 있다. 22:03에 발생한 이벤트라면 tumbling에서는 시각을 10분 단위로 내림한 22:00이 창의 시작이므로 창이 하나다. sliding에서는 이 시각을 포함하면서 시작이 5분의 배수인 10분 구간을 모두 찾으면 되므로 21:55-22:05와 22:00-22:10 두 개가 나온다. 슬라이드를 2분으로 줄이면 한 이벤트가 다섯 창에 속하고, 결과 행과 엔진 상태도 그만큼 늘어난다.

실습 6.1의 강남구 이벤트 7건(발생 22:01, 22:03, 22:04, 22:08, 22:11, 22:14, 22:24)을 두 방식으로 세면 차이가 수치로 드러난다.

방식	창별 건수	건수 합
tumbling (10분)	22:00-22:10 → 4, 22:10-22:20 → 2, 22:20-22:30 → 1	7
sliding (10분 창·5분 슬라이드)	21:55-22:05 → 3, 22:00-22:10 → 4, 22:05-22:15 → 3, 22:10-22:20 → 2, 22:15-22:25 → 1, 22:20-22:30 → 1	14

합계를 보면 tumbling은 7로 이벤트 수 그대로인데 sliding은 14, 즉 7의 두 배다. 슬라이드가 창 크기의 절반이라 모든 이벤트가 정확히 두 창에 세어졌기 때문이며, 이것은 오류가 아니라 sliding의 정의다. 따라서 **sliding 창의 건수를 창끼리 더하면 그 수는 이벤트 수가 아니다**. 감사에서 "대시보드는 14건인데 보고서는 7건"이라는 질문을 받지 않으려면, 집계표에 창 방식과 크기와 슬라이드를 함께 적어야 한다.

코드에서 두 방식의 차이는 인자 하나다.

```
F.window("event_time", "10 minutes")           # tumbling: 슬라이드 없음
F.window("event_time", "10 minutes", "5 minutes") # sliding: 5분마다 새 창
events.groupBy(F.window("event_time", "10 minutes"), "district").count()
```

경계 규칙도 함께 정해 둔다. 창은 시작을 포함하고 끝을 포함하지 않으므로 22:00-22:10 창은 [22:00:00, 22:10:00)이고, 정확히 22:10:00에 발생한 이벤트는 다음 창에 속한다. 사소해 보이지만 손으로 계산한 값과 엔진 결과가 1건 어긋나는 흔한 원인이 이 규칙이다.

5. watermark — 언제까지 기다릴지를 데이터로 정한다

이벤트 시간으로 집계하면 근본적인 긴장이 생긴다. 창을 빨리 확정하면 늦게 도착한 이벤트를 놓치고, 무한정 기다리면 아무것도 확정하지 못한 채 열린 창의 상태를 계속 들고 있어야 한다. watermark는 이 긴장을 선언 한 줄로 타협한다.

watermark는 관측된 최대 이벤트 시각에서 지연 허용값을 뺀 시각이다. "이보다 오래된 이벤트는 더 오지 않는다고 간주한다"는 기준선이며, 창의 끝이 이 기준선보다 과거가 되면 엔진은 창을 닫고 상태를 정리한 뒤 그 창에 오는 지연 이벤트를 폐기한다.

```
events.withWatermark("event_time", "10 minutes")
    .groupBy(F.window("event_time", "10 minutes"), "district").count()
```

여기서 반드시 붙잡아야 할 원리는 계산의 기준이 시계가 아니라는 점이다. "지금 시각 - 10분"이 아니라 "지금까지 본 이벤트 중 가장 최근 것의 이벤트 시각 - 10분"이므로, 데이터가 안 들어오면 watermark는 멈춰 있고 데이터가 몰려 들어오면 빠르게 전진한다. 실습에서 파일 1까지 관측한 최대 이벤트 시각이 22:09이므로 watermark는 21:59이고, 파일 2까지 보면 22:18이 최대라 22:08로, 파일 3까지 보면 22:24가 최대라 22:14로 전진한다.

이 규칙에 자연 이벤트 두 건을 넣어 보면 판정이 걸린다. ****수용된 이벤트(LATE_OK)****는 22:04에 발생해 파일 2로 도착했다. 도착 시점의 watermark는 22:09 - 10분 = 21:59이고, 이 이벤트가 속한 창 22:00-22:10의 끝은 21:59보다 미래이므로 창이 아직 열려 있다. 실측 결과 그 창의 강남구 건수가 **3에서 4로 갱신됐다**.

****폐기된 이벤트(LATE_DROP)****는 21:49에 발생해 파일 3으로 도착했다. 도착 시점의 watermark는 22:18 - 10분 = 22:08이고, 속한 창 21:40-21:50의 끝은 22:08보다 과거라 창이 이미 닫혔다. 실측 결과 어떤 창도 갱신되지 않았고, 엔진 메트릭(metric) `numRowsDroppedByWatermark` 가 **0에서 1로 증가했다**.

두 판정의 증거가 서로 다른 곳에 남는다는 점이 운영에서 중요하다. 수용은 집계 결과의 행 갱신으로 눈에 보이지만, **폐기는 집계 결과 어디에도 나타나지 않고 행이 없을 뿐이다**. 그래서 폐기를 보려면 집계가 아니라 엔진 메트릭을 감시해야 한다. 4장에서 커밋을 처리보다 앞세운 시나리오의 유실이 로그 없이 조용했던 것과 정확히 같은 구조다.

같은 이벤트라도 언제 도착했느냐에 따라 판정이 뒤집힌다는 것도 확인해 둘 만하다. LATE_OK가 한 파일 늦게 왔다면 그때는 watermark가 22:14로 전진해 창 22:00-22:10이 이미 닫힌 뒤였을 것이고, 판정은 수용에서 폐기로 바뀐다. 폐기 여부를 정하는 것은 "그 이벤트가 얼마나 늦었나"가 아니라 "엔진이 그사이 무엇을 봤나"다.

watermark는 폐기 규칙이면서 동시에 청소 규칙이다. watermark가 없으면 엔진은 21:40 창에도 언젠가 이벤트가 올 가능성 때문에 모든 창의 상태를 영원히 들고 있어야 한다. 그래서 자연 허용을 길게 잡는 결정은 자연 수용을 늘리는 동시에 상태를 오래 유지하는 결정이며, "넉넉하게 하루로 잡자"는 공짜가 아니다.

값은 임의로 정하지 않고 **실제 자연 분포를 관찰해 정한다**. 이벤트 시간과 도착 시간의 차이를 모아 중앙값과 상위 분위수와 최댓값을 계산하고, 그 꼬리를 뺄도록 잡는다(4장 실측은 평균 3.5ms, 최대 110.7ms로 로컬 환경이라 극히 짧다). 원천이 데이터를 30분 뒤에 게시하는 구조라면 그 30분이 하한이 되고, 상한은 하루 판단의 시급성이 정한다. 운영 중에는 폐기 카운터를 지표로 감시하다가 폐기가 늘면 자연 분포가 변했다는 신호로 읽고 값을 다시 계산한다.

6. 알림 — 집계 위의 필터에 함정이 셋

윈도우 집계가 만들어지면 알림은 그 위에 얹는 필터다. "10분 창에서 같은 지역구 민원이 N건 이상이면 알린다"처럼 한 줄로 쓸 수 있는데, 여기에 앞에서 본 문제가 모두 다시 등장한다.

첫째, 미확정 창에서 올리는 알림이다. update 모드에서 창의 수치는 자연 이벤트로 계속 자란다. 임계를 4건으로 두고 실습의 흐름을 따라가면, 파일 1을 처리한 시점에 강남구는 3이라 알림이 없고, 파일 2에서 자연분이 수용되어 4가 되는 순간 임계에 닿는다. 그런데 그 시점은 창이 끝난 22:10보다 한참 뒤다. 창이 확정된 뒤에 평가하기로 하면 판정이 뒤집히는 오경보는 없지만 창 크기에 watermark를 더한 만큼 더 늦는다. 빠른 알림과 정확한 알림은 같은 설계가 아니다.

둘째, 알림 중복이다. 갱신마다 조건을 재평가하면 같은 창이 조건을 두 번 만족해 4에서 한 번, 5에서 또 한 번 발화한다. 4장과 5장에서 본 중복 문제가 알림 계층에서 그대로 재현된 것이고, 해법도 같다. 창 시작 시각과 지역구와 규칙 ID를 묶어 중복 제거 키로 삼고, 발송 기록에 유니크 제약을 걸어 5장의 멍등 쓰기를 적용한다.

셋째, 침묵이다. 알림 조건은 "데이터가 있는데 많음"만 잡아내고, 데이터가 아예 안 들어오는 수집 장애는 어떤 창도 임계를 넘지 않으므로 조용하다. 더 까다로운 점은 집계 스트림에서 0건인 창이 건수 0인 행으로 나오는 것이 아니라 **행 자체가 없다**는 것이다. 이벤트가 없으면 창도 만들어지지 않으므로 "0건이면 경보"라는 조건을 집계 결과 위에 걸 수 없고, 일정 시간 결과 행이 하나도 안 나왔음을 감시하는 하트비트(heartbeat)나 유입량 하한 감시를 따로 두어야 한다(11장). 4장의 유실, 5절의 폐기에 이어 세 번째로 확인하는 같은 교훈이다.

공공에서는 임계를 정하는 방식에도 제약이 붙는다. 임계를 절대 건수로 잡으면 대도시만 늘 경보가 뜨고 소도시의 상대적 급증은 묻히므로, "평시 분포 대비 몇 배"로 잡아야 형평성이 지켜진다. 그리고 빠른 알림과 정확한 알림 중 무엇을 택했는지, 그 결정을 누가 승인했는지를 알림마다 문서로 남겨야 오발령이 났을 때 설계로 설명할 수 있다.

7. 실습 6.1 지역별 민원 이벤트 윈도우 집계 (난이도 ★★★★★)

실습 목표

- 파일 소스 스트림에 watermark와 tumbling window 집계를 구성하고 update 모드로 출력한다.
- 자연 이벤트 두 건(수용 1, 폐기 1)의 판정을 실측 증거로 확인해 자연 이벤트 처리 평가표를 만든다.

민원 이벤트 14건은 시뮬레이션 입력이다. 개인정보가 든 실제 민원을 쓸 수 없기 때문이며, 관찰 대상은 데이터 내용이 아니라 실제 엔진의 윈도우-watermark 동작이다. 판정은 사람이 단정하지 않고 스크립트가 집계 갱신과 폐기 카운터에서 계산해 기록한다.

시작 전 준비

PySpark는 **Java 17 이상**이 있어야 돈다. 아래 한 줄이 Java 버전을 확인하고 가상환경과 패키지를 준비한다.

```
python scripts/setup_practice.py 6
```

Java가 없거나 버전이 낮으면 무엇을 설치할지 알려 준다.

선수 조건과 실행

Java 17 이상이 필요하고(`JAVA_HOME` 설정), Python은 PySpark가 지원하는 버전을 쓴다(이 실습은 3.13에서 검증했다). 브로커 없이 파일 소스로 단독 실행한다.

```
cd practice/chapter6
python3.13 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter6.py
```

스크립트는 이벤트 파일 세 개를 스트림 입력 디렉터리에 순서대로 도착시킨다. 파일 1은 정시 6건(강남 22:01-22:03-22:08, 마포 22:02-22:09, 관악 22:05)으로 창 22:00-22:10의 초기 집계를 만든다. 파일 2는 정시 4건(강남 22:11-22:14, 마포 22:12-22:18)과 함께 LATE_OK(강남, 발생 22:04)를 넣어 수용을 관찰하고, 파일 3은 LATE_DROP(마포, 발생 21:49)과 정시 2건(관악 22:21, 강남 22:24)을 넣어 폐기를 관찰한다. 파일 3의 정시 2건이 watermark를 22:14까지 밀어 올려, 창 22:00-22:10이 닫히는 것까지 한 실행에서 확인된다.

핵심 코드

```
counts = (events.withWatermark("event_time", "10 minutes")
           .groupBy(F.window("event_time", "10 minutes"), "district").count())
query = counts.writeStream.outputMode("update").foreachBatch(sink).start()
query.processAllAvailable() # 파일 투입 후 배치 경계 고정
```

전체 코드는 `practice/chapter6/code/6-1-streaming-window-aggregation.py` 참고

실행 결과와 대조

micro-batch별로 바뀐 행만 나온다. 배치 0(파일 1)에서는 창 22:00-22:10에 강남구 3, 마포구 2, 관악구 1이 나오고, 배치 2(파일 2)에서는 창 22:00-22:10 강남구가 4로 다시 나오면서 창 22:10-22:20의 강남구 2와 마포구 2가 처음 등장한다. 배치 4(파일 3)에서는 창 22:20-22:30의 강남구 1과 관악구 1만 나오고, 21:40-

21:50 창은 어디에도 없다. 배치 1:3:5는 출력이 0행인데, 파일을 처리한 직후 추가 데이터를 확인하려고 도는 빈 배치이므로 micro-batch 번호와 입력 파일 번호가 일치하지 않는다.

배치 2의 세 행은 성격이 다르다. "22:00-22:10 강남구 4"는 지연 수용으로 인한 과거 창의 갱신이고, "22:10-22:20"의 두 행은 새 창의 첫 등장이다. update 모드는 둘을 구분하지 않고 바뀐 행으로 똑같이 내보내므로, 받는 쪽은 창과 지역구를 키로 삼은 업서트로 통일해 받는다.

표 6.4 지연 이벤트 처리 평가표(watermark 10분, 창 10분 — 실측. 실무교재와 같은 표 번호다)

이벤트	발생 시각	도착 시점 watermark	속한 창	판정	증거
LATE_OK	22:04	21:59 (= 22:09 - 10분)	22:00-22:10 (끝 > watermark)	수용	강남구 창 건수 3 → 4 갱신
LATE_DROP	21:49	22:08 (= 22:18 - 10분)	21:40-21:50 (끝 < watermark)	폐기	numRowsDroppedByWatermark 0 → 1

산출물은 세 개다. `ch6_window_counts.jsonl`은 micro-batch별 출력 행을 그대로 남긴 집계 기록이고, `ch6_late_event_report.json`은 위 평가표의 원본으로 판정과 증거와 폐기 카운터를 담은 검수 자산이며, `ch6_sliding_demo.json`은 같은 이벤트의 sliding 집계로 4절의 합계 7 대 14 비교의 근거다.

8. 핵심 요약

- 스트림은 끝없이 자라는 테이블이고, 배치 질의를 그 위에 그대로 얹는다.** 엔진은 중간 결과인 상태를 들고 증분으로 갱신하므로 전체를 다시 세지 않는다. trigger가 언제 계산할지를, output mode가 무엇을 내보낼지를 따로 정하며, update 출력은 같은 키가 반복되므로 받는 쪽에 5장의 업서트가 필요하다.
- 창 집계의 기준은 이벤트 시간이고, sliding 건수의 합은 이벤트 수가 아니다.** 같은 강남구 7건이 tumbling에서는 합계 7이지만 sliding(10분 창·5분 슬라이드)에서는 14로 나온다. 오류가 아니라 정의이므로, 집계표에 창 방식과 크기와 슬라이드를 병기하지 않으면 두 배로 부풀려 읽힌다.
- watermark는 관측된 최대 이벤트 시각에서 지연 허용을 뺀 값이다.** 시계가 아니라 데이터에서 계산되므로 유입이 멈추면 창도 닫히지 않는다. 22:04에 발생한 이벤트는 watermark 21:59 시점에 도착해 창이 열려 있었으므로 수용되어 집계가 3에서 4로 갱신됐고, 21:49에 발생한 이벤트는 watermark 22:08 시점에 도착해 창 21:40-21:50이 이미 닫혔으므로 폐기됐다.
- 수용과 폐기는 서로 다른 곳에 증거를 남긴다.** 수용은 집계 행의 갱신으로 보이지만 폐기는 결과에 흔적이 없고 `numRowsDroppedByWatermark`가 0에서 1로 오른 것으로만 확인된다. 이 카운터를 감시하지 않으면 지연 데이터가 사라진 사실 자체를 알 수 없다.
- 알림은 집계 위의 필터지만 미확정·중복·침묵을 함께 설계해야 한다.** 지연 수용으로 수치가 자라므로 빠른 알림과 정확한 알림은 다른 설계이고, 같은 창의 재발화는 중복 제거 키로 막는다. 데이터가 아예 안 들어오는 장애는 창 자체가 생기지 않아 조용하므로 별도 감시 장치가 필요하다.

9. 과제

실습을 실행해 생성된 다음 파일을 LMS에 그대로 업로드한다.

- `practice/chapter6/data/output/ch6_late_event_report.json`

이어서 본인 파일의 `watermark_delay`, `window_size`, 각 지연 이벤트의 `event_time` 과 `watermark_at_arrival` 과 `decision` 을 인용해 세 가지를 각각 세 문장 이내로 쓴다.

1. 수용된 이벤트와 폐기된 이벤트 각각에 대해, 도착 시점의 watermark가 어떻게 계산됐는지 산술로 재현한다(관측된 최대 이벤트 시각 - `watermark_delay`).
2. 폐기된 이벤트는 집계 산출물 어디에도 나타나지 않는다. 운영에서 이 폐기를 무엇으로 감지해야 하는지 쓴다.
3. "우리 대시보드 수치는 실시간이라 정확하다"는 문장을 본인의 `window_size` 와 폐기 판정을 근거로 반박한다.

수업일 포함 7일 이내에 제출한다.

10. 더 알아보기

이 장에서 다루지 않은 내용은 실무교재 `docs/chapter6.md` 에 있다.

- 네 가지 질문(**what-where-when-how**)과 배치 질의 대조표 — 창-trigger-output mode가 각각 어느 질문에 답하는지: `docs/chapter6.md` §6.1
- 세션 창과 창 방식의 세 번째 선택지 — 시계가 아니라 데이터의 공백이 창의 경계를 정하는 방식: §6.2
- Flink와의 대조, 여러 스트림을 조인할 때의 watermark — 느린 쪽 기준으로 창이 닫히는 문제: §6.3
- 체크포인트(checkpoint)와 상태 저장 연산의 메모리 비용 — 지연 허용을 늘리는 것이 곧 상태를 늘리는 이유: 보충 절
- 실행 환경 오류 사례 — 드라이버와 워커의 파이썬 버전 불일치와 `PYSPARK_PYTHON` 설정: 실습 6.1
- micro-batch별 출력 원본 — `practice/chapter6/data/output/ch6_window_counts.jsonl`